

16 Sort Aggregate vs Hash Aggregate

Aggregation in Spark at a high level

When you do operations like

- `groupBy`
- `agg`
- `count`
- `sum`
- `avg`

Spark has to **group rows by keys and aggregate values**.

Spark has **two main physical strategies** to do this:

1. Hash Aggregate
2. Sort Aggregate

You see these in the **physical plan** and Spark UI.

Hash Aggregate

What it does

- Spark creates an **in memory hash table**
- Key → aggregated values are stored in memory
- For each row:
 - Hash the key
 - Update aggregation buffer

Example mental model:

```
{ "Electronics" → 1500,
```

```
"Fashion"    → 900 }
```

When Spark prefers Hash Aggregate

Spark uses **Hash Aggregate** when:

- Group by keys fit in memory
- Data is not already sorted
- Hash map based aggregation is cheaper than sorting

This is the **default and fastest** approach in most cases.

Advantages

- Very fast for low to medium cardinality
- No sorting overhead
- Best for most real world aggregations

Limitations

- Memory dependent
- If hash table grows too large:
 - Spark may spill
 - Or switch strategy internally

How it appears in explain plan

```
HashAggregate
```

Often you will see **two HashAggregates**:

- Partial Hash Aggregate (before shuffle)
- Final Hash Aggregate (after shuffle)

This is normal and expected.

Sort Aggregate

What it does

- Spark **sorts data by group by key**
- Then scans sorted data
- Aggregates consecutive rows with same key

Example flow:

1. Sort by category
 2. Electronics, Electronics, Electronics
 3. Fashion, Fashion
 4. Aggregate while scanning
-

When Spark uses Sort Aggregate

Spark uses **Sort Aggregate** when:

- Data is already sorted by grouping key
- Or hash aggregation is not suitable
- Or Spark is forced to use sort based aggregation

Common triggers:

- Very high cardinality keys
 - Memory pressure
 - Hash aggregate disabled
-

Advantages

- Lower memory usage
 - Stable for very large number of groups
 - Works even when hash tables cannot fit in memory
-

Limitations

- Sorting is expensive
- Slower than hash aggregate in most cases
- Shuffle + sort overhead

How it appears in explain plan

SortAggregate

You will always see a **Sort** operator before it.

Hash vs Sort Aggregate summary table

Aspect	Hash Aggregate	Sort Aggregate
Technique	Hash table	Sorting + scan
Memory usage	High	Lower
Speed	Faster	Slower
Needs sorting	No	Yes
Default choice	Yes	No